

Functional Programming Patterns

Functional Programming (FP) হলো এমন একটি programming style যেখানে computation-কে মূলত **functions**-এর মাধ্যমে **express** করা হয়, এবং predictable, composable ও reusable code লেখার দিকে focus করা হয়।

Topics Covered

1. Pure Functions & Side Effects
 2. Immutability Patterns — Spread, `Object.freeze()`
 3. Higher-Order Functions
 4. Function Composition — `compose()` & `pipe()`
 5. Currying & Partial Application
 6. Array Method Chaining
 7. Transducers
-

1. Functional Programming কী?

Functional Programming হলো এমন একটি programming paradigm যেখানে function-কে first-class value হিসেবে ব্যবহার করে program-এর logic তৈরি করা হয়।

JavaScript functional programming-এর জন্য খুব ভালো language কারণ JavaScript-এ function:

- variable-এ রাখা যায়
- argument হিসেবে pass করা যায়
- return করা যায়
- অন্য function-এর ভিতরে তৈরি করা যায়
- array methods-এর সাথে ব্যবহার করা যায়

Example:

```
const add = (a, b) => a + b;  
  
console.log(add(10, 20));
```

এখানে computation একটি function-এর মাধ্যমে করা হচ্ছে।

2. Imperative vs Functional Style

Imperative

Imperative programming-এ আমরা বলি:

কীভাবে কাজটা করতে হবে?

```
const numbers = [1, 2, 3, 4, 5];

const doubled = [];

for (const number of numbers) {
  doubled.push(number * 2);
}

console.log(doubled);
```

এখানে step-by-step instructions দেওয়া হচ্ছে।

Functional

Functional style:

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(number => number * 2);

console.log(doubled);
```

এখানে আমরা বলছি:

প্রতিটি number-কে double করো।

3. Functional Programming-এর Core Ideas

Functional Programming-এর কিছু গুরুত্বপূর্ণ ধারণা:

Pure Functions
↓
Immutability
↓
Higher-Order Functions
↓
Composition
↓
Currying / Partial Application
↓
Chaining
↓
Advanced transformations
↓
Transducers

সবগুলো একে অপরের সাথে connected ।

4. Pure Functions

Pure Function কী?

একটি function pure হলে সাধারণভাবে দুটি condition satisfy করে:

1. Same input → Same output

```
function add(a, b) {  
  return a + b;  
}
```

```
add(2, 3); // 5  
add(2, 3); // 5  
add(2, 3); // 5
```

একই input দিলে সবসময় একই output ।

2. No observable side effects

Function-এর বাইরে থাকা state পরিবর্তন করবে না ।

```
function add(a, b) {  
  return a + b;  
}
```

এখানে বাইরের কিছু পরিবর্তন হচ্ছে না।

তাই এটি pure।

5. Impure Function

Example:

```
let total = 0;  
  
function addToTotal(value) {  
  total += value;  
}
```

এখানে function বাইরের variable `total` পরিবর্তন করছে।

তাই functionটি impure।

6. Pure vs Impure

Pure	Impure
Same input → same output	Output external state-এর উপর depend করতে পারে
No observable side effect	Side effect থাকতে পারে
Easy to test	তুলনামূলক কঠিন
Predictable	Less predictable
সহজে compose করা যায়	Composition কঠিন হতে পারে

7. Side Effect কী?

Side effect হলো function-এর computation-এর বাইরে observable কোনো পরিবর্তন বা interaction।

Common side effects:

- global variable পরিবর্তন
- object mutation
- array mutation
- DOM manipulation
- API request
- database operation
- file write
- logging
- localStorage
- random value generation
- current time read করা
- external system-এর সাথে interaction

Example

```
function greet(name) {  
  console.log(`Hello ${name}`);  
  
  return `Hello ${name}`;  
}
```

Return value predictable হলেও `console.log()` একটি side effect ।

8. সব Side Effect খারাপ?

না।

বাস্তব application-এ side effect দরকার।

যেমন:

```
fetch("/api/users");
```

API call side effect-এর category-তে পড়ে।

Problem হলো:

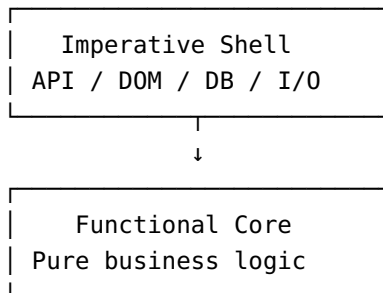
Side effect-কে uncontrolled ভাবে ছড়িয়ে দেওয়া।

Better architecture:

```
Pure business logic
  ↓
Side-effect boundary
  ↓
API / DB / DOM / Storage
```

9. Functional Core, Imperative Shell

একটি useful architecture:



Example:

```
function calculateTotal(items) {
  return items.reduce(
    (total, item) => total + item.price,
    0
  );
}
```

এটি pure ।

তারপর:

```
const items = await fetchItems();

const total = calculateTotal(items);
```

```
console.log(total);
```

API call impure boundary-তে থাকছে, calculation pure থাকছে।

10. Immutability

Immutability কী?

Immutability মানে existing data directly modify না করে নতুন data তৈরি করা।

Mutable approach:

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
user.age = 32;
```

এখানে original object mutate হয়েছে।

11. Immutable Update with Spread

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
const updatedUser = {  
  ...user,  
  age: 32  
};
```

এখন:

```
console.log(user.age);  
// 31
```

```
console.log(updatedUser.age);  
// 32
```

Original object unchanged |

12. Immutable Array Update

Wrong:

```
const numbers = [1, 2, 3];  
  
numbers.push(4);
```

Immutable approach:

```
const numbers = [1, 2, 3];  
  
const updatedNumbers = [  
  ...numbers,  
  4  
];
```

13. Remove Item Immutably

```
const numbers = [10, 20, 30, 40];  
  
const updatedNumbers = numbers.filter(  
  number => number !== 30  
);
```

Original:

```
[10, 20, 30, 40]
```

New:

```
[10, 20, 40]
```

14. Update Array Item Immutably

```
const users = [
  { id: 1, name: "Rahim" },
  { id: 2, name: "Karim" }
];

const updatedUsers = users.map(user =>
  user.id === 2
    ? { ...user, name: "Hasan" }
    : user
);
```

এখানে:

- array নতুন
- updated object নতুন
- untouched object reference রাখা হয়েছে

15. Nested Immutable Update

```
const user = {
  name: "Shariar",
  address: {
    city: "Rangpur"
  }
};
```

Update:

```
const updatedUser = {
  ...user,
  address: {
    ...user.address,
    city: "Dhaka"
  }
};
```

```
}  
};
```

এখানে nested object-ও নতুন তৈরি হয়েছে।

16. Object.freeze()

`Object.freeze()` object-এর properties-কে modification থেকে protect করতে সাহায্য করে।

```
const user = {  
  name: "Sharir",  
  age: 31  
};  
  
Object.freeze(user);  
  
user.age = 32;  
  
console.log(user.age);
```

Non-strict mode-এ assignment silently fail করতে পারে।

Strict mode-এ error হতে পারে।

17. Important: Object.freeze() Shallow

এটা খুব important।

```
const user = {  
  name: "Sharir",  
  address: {  
    city: "Rangpur"  
  }  
};  
  
Object.freeze(user);  
  
user.address.city = "Dhaka";
```

Nested object freeze হয়নি।

কারণ:

```
user
  ↓
address —→ nested object
```

`Object.freeze(user)` শুধু top-level object freeze করেছে।

18. Deep Immutability

Deep immutability দরকার হলে nested structures-ও freeze করতে হবে।

Conceptually:

```
function deepFreeze(object) {
  Object.freeze(object);

  for (const value of Object.values(object)) {
    if (
      value &&
      typeof value === "object" &&
      !Object.isFrozen(value)
    ) {
      deepFreeze(value);
    }
  }

  return object;
}
```

তবে production application-এ সব object deep-freeze করা performance-এর জন্য সবসময় ভালো idea নয়।

অনেক সময় immutable update pattern ব্যবহার করাই practical।

19. Immutability কেন Useful?

Immutability-এর সুবিধা:

Predictability

আপের data হঠাৎ পরিবর্তন হয়ে যাওয়ার possibility কমে।

Debugging

Old state preserve করা সহজ।

State management

React/Redux-এর মতো ecosystem-এ খুব useful।

Time travel

আপের state ধরে রাখা সহজ।

Referential equality

নতুন reference দেখে change detect করা সহজ।

20. Higher-Order Functions

HOF কী?

Higher-Order Function হলো এমন function যেটা:

1. একটি বা একাধিক function argument হিসেবে নেয়

অথবা

1. একটি function return করে

অথবা দুটোই করে।

21. Function as Argument

```
function applyOperation(a, b, operation) {  
  return operation(a, b);  
}  
  
const add = (a, b) => a + b;
```

```
console.log(
  applyOperation(10, 20, add)
);
```

এখানে:

```
operation
```

একটি function ।

তাই `applyOperation()` হলো Higher-Order Function ।

22. Function Returning Function

```
function multiplier(factor) {
  return function (number) {
    return number * factor;
  };
}
```

Use:

```
const double = multiplier(2);

console.log(double(5));
// 10
```

এখানে outer function একটি function return করছে ।

23. Built-in Higher-Order Functions

JavaScript-এর অনেক array methods HOF:

```
map()
filter()
reduce()
find()
some()
```

```
every()  
sort()
```

Example:

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.map(  
  number => number * 2  
);
```

`map()` callback function গ্রহণ করছে।

24. Callback vs Higher-Order Function

এই distinction গুরুত্বপূর্ণ।

```
numbers.map(number => number * 2);
```

এখানে:

```
map()  
↓  
Higher-Order Function
```

আর:

```
number => number * 2
```

হলো:

Callback Function

অর্থাৎ callback হলো function যা অন্য function-এ pass করা হয়।

Higher-order function হলো function যা function গ্রহণ করে/return করে।

25. Function Composition

Composition কী?

ছোট ছোট function combine করে বড় function তৈরি করাকে function composition বলা হয়।

ধরো:

```
const double = x => x * 2;  
  
const square = x => x * x;
```

Manual:

```
const result = square(double(5));  
  
console.log(result);
```

Flow:

```
5  
↓  
double  
↓  
10  
↓  
square  
↓  
100
```

26. `compose()`

একটি সাধারণ `compose()` function:

```
const compose = (f, g) => x =>  
  f(g(x));
```

Example:

```
const double = x => x * 2;

const square = x => x * x;

const doubleThenSquare = compose(
  square,
  double
);

console.log(doubleThenSquare(5));
```

Result:

100

কারণ:

```
compose(square, double)
      ↓
square(double(x))
```

27. Composition Right-to-Left

Traditional `compose()` সাধারণত:

```
compose(f, g)
      ↓
f(g(x))
```

অর্থাৎ:

right → left

Example:

```
const addOne = x => x + 1;
const double = x => x * 2;
```

```
const fn = compose(
  addOne,
  double
);

fn(5);
```

Flow:

```
5
↓
double
↓
10
↓
addOne
↓
11
```

28. pipe()

`pipe()` composition-এর বিপরীত reading direction ব্যবহার করে।

```
const pipe = (f, g) => x =>
  g(f(x));
```

Example:

```
const addOne = x => x + 1;

const double = x => x * 2;

const fn = pipe(
  addOne,
  double
);

console.log(fn(5));
```

Result:

12

Flow:

```
5
↓
addOne
↓
6
↓
double
↓
12
```

29. `compose()` vs `pipe()`

	<code>compose()</code>	<code>pipe()</code>
Direction	Right → Left	Left → Right
Example	<code>f(g(x))</code>	<code>g(f(x))</code>
Reading	Mathematical	Sequential
Mental model	Inside-out	Step-by-step

30. Variadic `compose()`

Practical version:

```
const compose = (...functions) => value =>
  functions.reduceRight(
    (acc, fn) => fn(acc),
    value
  );
```

Example:

```
const addOne = x => x + 1;
const double = x => x * 2;
const square = x => x * x;

const process = compose(
  square,
  double,
  addOne
);

console.log(process(3));
```

Flow:

```
3
↓
addOne
↓
4
↓
double
↓
8
↓
square
↓
64
```

31. Variadic `pipe()`

```
const pipe = (...functions) => value =>
  functions.reduce(
    (acc, fn) => fn(acc),
    value
  );
```

Example:

```
const process = pipe(
  addOne,
  double,
```

```
    square
  );

  console.log(process(3));
```

Flow:

```
3
↓
addOne
↓
4
↓
double
↓
8
↓
square
↓
64
```

32. Why Composition Useful?

ধরো:

```
const trim = text => text.trim();

const lowercase = text => text.toLowerCase();

const normalize = pipe(
  trim,
  lowercase
);
```

এখন:

```
normalize("  HELLO WORLD  ");
```

Result:

```
hello world
```

ছোট reusable function দিয়ে complex transformation তৈরি করা যায়।

33. Currying

Currying কী?

একটি multi-argument function-কে এমন series of functions-এ transform করা যেখানে প্রতিটি function একবারে একটি argument গ্রহণ করে।

Normal:

```
function add(a, b, c) {  
  return a + b + c;  
}
```

Curried:

```
function add(a) {  
  return function (b) {  
    return function (c) {  
      return a + b + c;  
    };  
  };  
}
```

Use:

```
console.log(  
  add(10)(20)(30)  
);
```

Result:

```
60
```

34. Arrow Function Currying

```
const add = a => b => c =>  
  a + b + c;
```

Use:

```
add(10)(20)(30);
```

35. Why Currying?

Currying-এর সবচেয়ে বড় benefit হলো **partial application** সহজ করা।

Example:

```
const multiply = a => b =>  
  a * b;
```

এখন:

```
const double = multiply(2);
```

এখন:

```
double(10); // 20  
double(50); // 100  
double(100); // 200
```

2 already fixed।

36. Currying vs Partial Application

দুটো related কিন্তু exactly same নয়।

Currying

Multi-argument function:

```
f(a, b, c)
```

কে:

```
f(a)(b)(c)
```

এ convert করে।

Partial Application

কিছু arguments আগে থেকেই fix করে নতুন function তৈরি করা।

Example:

```
function multiply(a, b, c) {  
  return a * b * c;  
}  
  
const multiplyByTwo = (b, c) =>  
  multiply(2, b, c);
```

এখানে প্রথম argument `2` fixed।

37. `bind()` দিয়ে Partial Application

```
function multiply(a, b, c) {  
  return a * b * c;  
}  
  
const multiplyByTwo = multiply.bind(null, 2);  
  
console.log(  
  multiplyByTwo(3, 4)  
);
```

Result:

24

এখানে 2 আগে থেকেই fixed ।

38. Generic Curry Function

একটি basic generic curry utility:

```
function curry(fn) {  
  return function curried(...args) {  
    if (args.length >= fn.length) {  
      return fn(...args);  
    }  
  
    return (...nextArgs) =>  
      curried(...args, ...nextArgs);  
  };  
}
```

Example:

```
function add(a, b, c) {  
  return a + b + c;  
}  
  
const curriedAdd = curry(add);  
  
console.log(  
  curriedAdd(10)(20)(30)  
);
```

এমন utility functional programming libraries-এ common concept ।

39. Array Method Chaining

Array methods একটার result-এর উপর আরেকটা method call করা যায় ।

Example:

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers
  .filter(number => number % 2 === 0)
  .map(number => number * 10);
```

Flow:

```
[1,2,3,4,5,6]
  ↓
filter
  ↓
[2,4,6]
  ↓
map
  ↓
[20,40,60]
```

40. Longer Chain

```
const result = users
  .filter(user => user.active)
  .map(user => user.name)
  .map(name => name.toUpperCase())
  .sort();
```

এখানে প্রতিটি method আগের result-এর উপর কাজ করছে।

41. filter() → map() → reduce()

এটি খুব common FP pattern।

```
const total = products
  .filter(product => product.active)
  .map(product => product.price)
  .reduce(
```

```
(sum, price) => sum + price,  
0  
);
```

Flow:

```
products  
↓  
filter  
↓  
active products  
↓  
map  
↓  
prices  
↓  
reduce  
↓  
total
```

42. Chaining-এর সুবিধা

Readability

Transformation pipeline-এর মতো দেখা যায়।

Reusability

প্রতিটি callback ছোট রাখা যায়।

Declarative style

কী result চাই সেটা বোঝা সহজ।

43. Chaining-এর সমস্যা

অতিরিক্ত chain readability কমাতে পারে।

Bad:

```
const result = data
  .filter(...)
  .map(...)
  .filter(...)
  .map(...)
  .filter(...)
  .reduce(...)
  .map(...)
  .filter(...);
```

এটা technically কাজ করতে পারে, কিন্তু বুঝতে কঠিন।

Better:

```
const activeUsers = data.filter(...);

const names = activeUsers.map(...);

const result = names.filter(...);
```

অথবা reusable functions:

```
const getActiveUsers = users =>
  users.filter(user => user.active);

const getNames = users =>
  users.map(user => user.name);
```

তারপর:

```
const result = pipe(
  getActiveUsers,
  getNames
)(users);
```

44. Intermediate Arrays

একটি গুরুত্বপূর্ণ performance concept:

```
numbers
  .filter(...)
  .map(...)
  .filter(...);
```

প্রতিটি method সাধারণত intermediate array তৈরি করে।

Conceptually:

```
original array
  ↓
filter
  ↓
new array
  ↓
map
  ↓
new array
  ↓
filter
  ↓
new array
```

ছোট/medium datasets-এ সাধারণত সমস্যা নয়।

কিন্তু huge data বা performance-critical pipeline-এ এটা গুরুত্বপূর্ণ হতে পারে।

এখান থেকেই **Transducer** concept আসে।

45. Transducers

Transducer কী?

Transducer হলো এমন একটি abstraction যা transformation logic-কে intermediate collection তৈরি না করে এক transformation pipeline-এ compose করতে দেয়।

সহজভাবে:

```
Normal chaining:

Array
```

```
↓  
filter  
↓  
temporary array  
↓  
map  
↓  
temporary array  
↓  
reduce
```

Transducer approach:

```
Array  
↓  
filter + map  
↓  
single reduction
```

46. কেন Transducer দরকার?

ধরো:

```
const result = numbers  
  .filter(isEven)  
  .map(double)  
  .filter(isGreaterThanTen)  
  .reduce(sum, 0);
```

এখানে intermediate arrays তৈরি হয়।

Transducer transformation-গুলোকে combine করে final reducing operation-এর সাথে apply করতে পারে।

47. Reducer কী?

Transducer বুঝতে reducer concept বুঝতে হবে।

```
const sumReducer = (acc, value) =>
  acc + value;
```

এখানে:

```
acc = accumulated result
value = current item
```

Example:

```
[1, 2, 3].reduce(
  (acc, value) => acc + value,
  0
);
```

Flow:

```
0 + 1 = 1
1 + 2 = 3
3 + 3 = 6
```

48. Transformation vs Reduction

ধরো:

```
const double = number =>
  number * 2;
```

এটা transformation ।

আর:

```
const sum = (acc, value) =>
  acc + value;
```

এটা reduction ।

Transducer মূলত transformation logic-কে reducer-এর উপর apply করার abstraction ।

49. Basic Transducer Idea

ধরো আমরা চাই:

```
filter even
  ↓
double
  ↓
sum
```

Normal:

```
const result = numbers
  .filter(isEven)
  .map(double)
  .reduce(sum, 0);
```

Transducer mindset:

```
transformer
  ↓
reducer
  ↓
single traversal
```

50. Simple Transducer Implementation

একটি educational implementation:

```
const mapT = fn => reducer =>
  (acc, value) =>
    reducer(acc, fn(value));

const filterT = predicate => reducer =>
  (acc, value) =>
    predicate(value)
```

```
    ? reducer(acc, value)
    : acc;
```

এগুলো transformation তৈরি করছে।

51. Compose Transducers

```
const compose = (...functions) => value =>
  functions.reduceRight(
    (acc, fn) => fn(acc),
    value
  );
```

Transducers:

```
const transformation = compose(
  filterT(number => number % 2 === 0),
  mapT(number => number * 2)
);
```

52. Apply to Reducer

```
const sumReducer = (acc, value) =>
  acc + value;

const transformedReducer =
  transformation(sumReducer);
```

তারপর:

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers.reduce(
  transformedReducer,
  0
);
```

```
console.log(result);
```

Flow:

1 → ignored

2

↓

4

↓

sum

3 → ignored

4

↓

8

↓

sum

5 → ignored

6

↓

12

↓

sum

Result:

24

কারণ:

2 + 4 + 6

↓

4 + 8 + 12

↓

24

53. Why Transducer Can Be Efficient

Normal pipeline:

```
filter
↓
array
↓
map
↓
array
↓
reduce
```

Transducer:

```
input
↓
filter
↓
map
↓
reducer
```

একটি single reduction-এর মধ্যে transformations apply করা যায়।

এতে intermediate collection allocation কমেতে পারে।

54. Transducer vs Array Chaining

	Array Chaining	Transducer
Readability	সাধারণত বেশি	কম
Simplicity	Easy	Advanced
Intermediate arrays	থাকতে পারে	এড়ানো যায়
Learning curve	Low	High
Small data	Excellent	Often unnecessary
Large pipelines	May allocate more	More efficient potential

	Array Chaining	Transducer
Typical JS code	Very common	Specialized

55. কখন Transducer ব্যবহার করবে?

প্রতিটি `map().filter()` chain-কে transducer বানানোর দরকার নেই।

Transducer useful যখন:

- অনেক transformations compose করতে হচ্ছে
- intermediate collections expensive
- large datasets
- performance-sensitive pipelines
- reusable transformation pipeline দরকার
- functional programming architecture ব্যবহার করছ

Normal application code-এ:

```
filter()
map()
reduce()
```

বেশিরভাগ সময় যথেষ্ট।

56. Pure Functions + Immutability + HOF

এই তিনটা একসাথে Functional Programming-এর foundation।

```
Pure Functions
  +
Immutability
  +
Higher-Order Functions
  ↓
Predictable transformations
```

Example:

```
const getActiveUsers = users =>
  users.filter(user => user.active);

const getNames = users =>
  users.map(user => user.name);
```

দুটোই pure এবং composable ।

57. Composition + Currying

Currying এবং composition একসাথে খুব powerful ।

```
const multiply = a => b =>
  a * b;

const add = a => b =>
  a + b;

const double = multiply(2);
const addTen = add(10);
```

তারপর:

```
const process = pipe(
  double,
  addTen
);
```

Use:

```
process(5);
```

Flow:

```
5
↓
double
↓
10
↓
```

```
addTen  
↓  
20
```

58. Functional Pipeline Example

ধরো users:

```
const users = [  
  { name: "Rahim", age: 25, active: true },  
  { name: "Karim", age: 17, active: true },  
  { name: "Hasan", age: 30, active: false },  
  { name: "Nabil", age: 28, active: true }  
];
```

Reusable functions:

```
const getActiveUsers = users =>  
  users.filter(user => user.active);  
  
const getAdults = users =>  
  users.filter(user => user.age >= 18);  
  
const getNames = users =>  
  users.map(user => user.name);  
  
const sortNames = names =>  
  [...names].sort();
```

Pipeline:

```
const processUsers = pipe(  
  getActiveUsers,  
  getAdults,  
  getNames,  
  sortNames  
);  
  
console.log(  
  processUsers(  
    users  
  )  
);
```

```
processUsers(users)
);
```

Result:

```
[
  "Nabil",
  "Rahim"
]
```

59. Why [...names].sort()?

`sort()` original array mutate করে।

```
names.sort();
```

এটা mutation।

Functional style-এ:

```
[...names].sort();
```

নতুন array তৈরি করে sort করা হচ্ছে।

এটাই immutability pattern।

60. Functional Programming in React

React development-এ FP concepts খুব useful।

Bad mutation

```
state.users.push(newUser);

setState(state);
```

Better:

```
setState(prev => ({
  ...prev,
  users: [
    ...prev.users,
    newUser
  ]
}));
```

61. React State Transformation

```
setUsers(prevUsers =>
  prevUsers.map(user =>
    user.id === id
      ? { ...user, active: true }
      : user
  )
);
```

এখানে:

- mutation নেই
- নতুন array তৈরি হচ্ছে
- updated object নতুন
- transformation declarative

62. Redux এবং Immutability

Redux-style state updates-এ immutability গুরুত্বপূর্ণ।

Example:

```
return {
  ...state,
  count: state.count + 1
};
```

Nested update:

```
return {
  ...state,
  user: {
    ...state.user,
    name: "Sharlar"
  }
};
```

63. Common Mistakes

Mistake 1 — Pure function-এর ভিতরে mutation

```
function addUser(users, user) {
  users.push(user);

  return users;
}
```

এখানে input mutate হচ্ছে।

Better:

```
function addUser(users, user) {
  return [
    ...users,
    user
  ];
}
```

Mistake 2 — Spread মানেই deep immutability ভাবা

```
const copy = {
  ...original
};
```

এটা shallow copy।

Nested object shared থাকতে পারে।

Mistake 3 — `Object.freeze()` **deep freeze** মনে করা

```
Object.freeze(object);
```

শুধু shallow freeze করে।

Mistake 4 — **Currying** এবং **partial application** একই মনে করা

Currying:

```
f(a, b, c)
↓
f(a)(b)(c)
```

Partial application:

```
f(a, b, c)
↓
fix a
↓
new function(b, c)
```

Mistake 5 — **Composition order** ভুল করা

```
compose(f, g)
```

সাধারণত:

```
f(g(x))
```

আর:

```
pipe(f, g)
```

সাধারণত:

```
g(f(x))
```

Mistake 6 — অতিরিক্ত chaining

Readable code-এর বদলে যদি chain puzzle হয়ে যায়, তাহলে chain ভেঙে reusable functions তৈরি করো।

Mistake 7 — সব জায়গায় Transducer ব্যবহার করা

Transducer advanced optimization/pattern।

Simple:

```
filter().map().reduce()
```

প্রয়োজনে সেটাই ব্যবহার করো।

64. Functional Programming Decision Guide

Data transform করতে হবে?

```
map()  
filter()  
reduce()
```

Function argument হিসেবে **function** নিতে হবে?

```
Higher-Order Function
```

ছোট **functions combine** করতে হবে?

```
compose()  
pipe()
```

কিছু **arguments** আগে **fix** করতে হবে?

Partial Application

Multi-argument function → nested unary functions?

Currying

Existing data modify না করে **update**?

Immutability

Many transformations + intermediate arrays avoid করতে হবে?

Transducer

65. Output Prediction

Question 1

```
const numbers = [1, 2, 3];

const result = numbers
  .map(x => x * 2)
  .filter(x => x > 2);

console.log(result);
```

Output:

[4, 6]

Question 2

```
const add = a => b => a + b;  
  
const addTen = add(10);  
  
console.log(addTen(5));
```

Output:

15

Question 3

```
const pipe = (...fns) => value =>  
  fns.reduce((acc, fn) => fn(acc), value);  
  
const addOne = x => x + 1;  
const double = x => x * 2;  
  
const result = pipe(  
  addOne,  
  double  
) (5);  
  
console.log(result);
```

Output:

12

66. Practice Problems

Level 1 — Pure Functions

Problem 1

একটি pure function তৈরি করো:

```
calculateDiscount(price, percentage)
```

যেটা discount price return করবে।

Problem 2

একটি function তৈরি করো:

```
isEven(number)
```

যেটা pure হবে।

Level 2 — Immutability

Problem 3

```
const user = {  
  name: "Rahim",  
  age: 25  
};
```

Original object mutate না করে age করো।

Problem 4

```
const numbers = [1, 2, 3, 4];
```

Original array mutate না করে add করো।

Level 3 — HOF

Problem 5

একটি function তৈরি করো:

```
applyOperation(a, b, operation)
```

যেটা add/subtract/multiply করতে পারবে।

Problem 6

একটি function তৈরি করো:

```
createMultiplier(factor)
```

যেটা নতুন multiplier function return করবে।

Example:

```
const triple = createMultiplier(3);  
  
triple(10);  
// 30
```

Level 4 — Composition

Problem 7

এই functions:

```
const trim = text => text.trim();  
  
const lowercase = text => text.toLowerCase();  
  
const addPrefix = text => `User: ${text}`;
```

`pipe()` দিয়ে একটা pipeline তৈরি করো।

Expected:

```
" SHARIAR "
```

↓

```
"user: shariar"
```

● Level 5 — Currying

Problem 8

Normal:

```
function multiply(a, b, c) {  
  return a * b * c;  
}
```

এটাকে curried function বানাও।

Expected:

```
multiply(2)(3)(4);
```

Result:

```
24
```

● Level 6 — Chaining

Problem 9

```
const products = [  
  { name: "Laptop", price: 80000, active: true },  
  { name: "Phone", price: 50000, active: true },  
  { name: "Mouse", price: 2000, active: false }  
];
```

একটি chain তৈরি করো যেটা:

1. active products নেয়
2. price `10000` -এর বেশি নেয়
3. product names নেয়
4. uppercase করে

Expected:

```
[  
  "LAPTOP",  
  "PHONE"  
]
```

Level 7 — Advanced

Problem 10 — Build `pipe()`

নিজে `pipe()` implement করো:

```
const pipe = (...functions) => {  
  // implement  
};
```

তারপর:

```
const result = pipe(  
  x => x + 1,  
  x => x * 2,  
  x => x - 3  
) (5);
```

Level 8 — Transducer Challenge

এই pipeline:

```
numbers  
  .filter(x => x % 2 === 0)
```

```
.map(x => x * 2)
.reduce((sum, x) => sum + x, 0);
```

কে transducer-style single reduction-এ implement করার চেষ্টা করো।

Interview Questions

Beginner

1. Functional Programming কী?
2. Pure function কী?
3. Side effect কী?
4. Pure এবং impure function-এর difference কী?
5. Immutability কী?
6. Spread operator কীভাবে immutable update করতে সাহায্য করে?
7. `Object.freeze()` কী?
8. `Object.freeze()` কি deep freeze করে?

Intermediate

1. Higher-Order Function কী?
2. Callback এবং Higher-Order Function-এর difference কী?
3. Function composition কী?
4. `compose()` এবং `pipe()` -এর difference কী?
5. Currying কী?
6. Partial application কী?
7. Currying এবং partial application-এর difference কী?
8. Array method chaining কী?
9. Array chaining-এ intermediate arrays কেন তৈরি হতে পারে?

Advanced

1. Transducer কী?
2. Transducer কেন ব্যবহার করা হয়?
3. Transducer এবং `map().filter().reduce()` pipeline-এর difference কী?
4. Functional Core, Imperative Shell কী?
5. React state management-এ immutability কেন গুরুত্বপূর্ণ?
6. Referential equality কীভাবে immutable updates-এর সাথে related?
7. সব side effect কি avoid করা সম্ভব?
8. Functional programming-এর সুবিধা এবং limitations কী?

Quick Revision

Pure Function

```
const add = (a, b) => a + b;
```

Same input:

→ same output

No observable external mutation.

Immutability

```
const updated = {  
  ...original,  
  age: 32  
};
```

Original পরিবর্তন হয় না।

Higher-Order Function

```
const double = numbers =>  
  numbers.map(x => x * 2);
```

Function অন্য function ব্যবহার করছে।

Composition

```
compose(f, g)
```

মনে সাধারণভাবে:

```
f(g(x))
```

Pipe

```
pipe(f, g)
```

মানে:

```
g(f(x))
```

Currying

```
const add = a => b => c =>  
  a + b + c;
```

Use:

```
add(1)(2)(3);
```

Partial Application

```
const multiplyByTwo =  
  multiply.bind(null, 2);
```

কিছু argument আগে fixed ।

Chaining

```
data  
  .filter(...)  
  .map(...)  
  .reduce(...);
```

Transducer

Multiple transformations

↓

Compose

↓

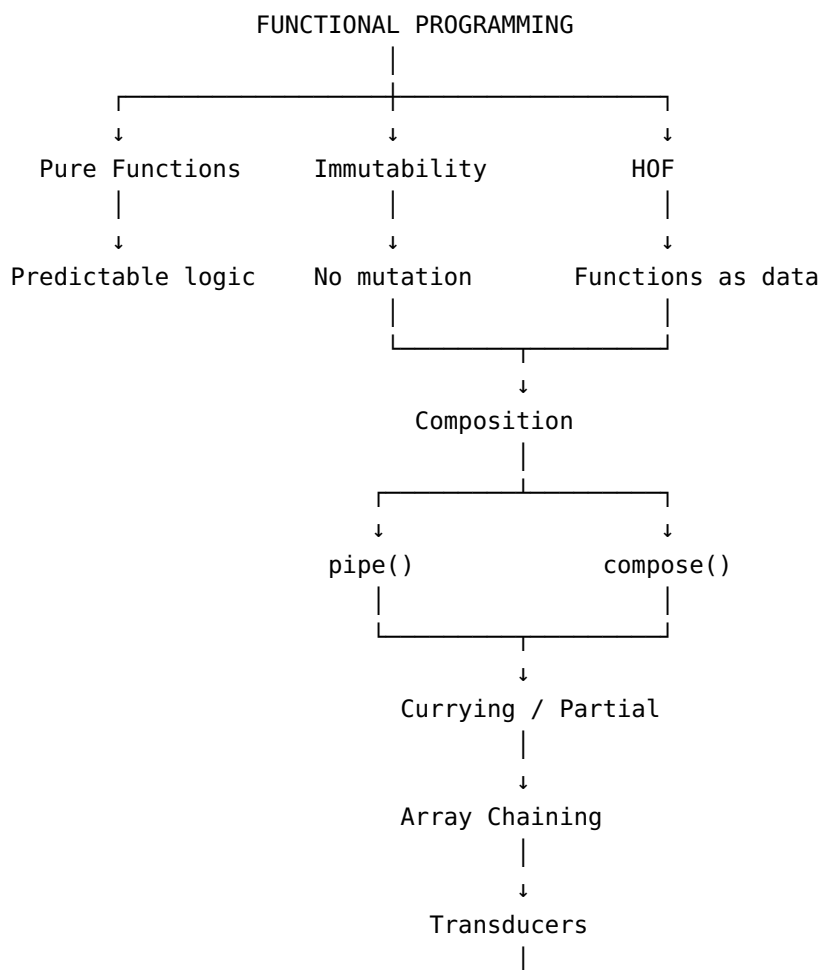
Single reduction

↓

Less intermediate collections

Final Mental Model

Functional Programming-এর পুরো section-টা এইভাবে মনে রাখো:





Functional Programming Rules to Remember

Rule 1

Prefer pure functions for business logic.

Rule 2

Don't mutate data unnecessarily.

Rule 3

Use HOFs to make behavior reusable.

Rule 4

Compose small functions instead of creating giant functions.

Rule 5

Use `pipe()` when you want a left-to-right readable pipeline.

Rule 6

Use currying when it genuinely improves reuse/composition.

Rule 7

Array chaining is powerful, but don't sacrifice readability.

Rule 8

Don't use `Object.freeze()` assuming it provides deep immutability.

Rule 9

Don't introduce transducers unless the complexity/performance trade-off is justified.

Rule 10

Functional Programming doesn't mean "never use side effects."

The goal is:

Control side effects
+
Keep transformations predictable
+
Make code composable
+
Avoid unnecessary mutation

✓ Mastery Checklist

- [] Functional Programming-এর basic philosophy বুঝি
- [] Pure function identify করতে পারি
- [] Side effect identify করতে পারি
- [] Pure vs impure explain করতে পারি
- [] Immutability বুঝি
- [] Spread দিয়ে immutable object update করতে পারি
- [] Spread দিয়ে immutable array update করতে পারি
- [] Nested immutable update করতে পারি
- [] `Object.freeze()` এবং shallow freeze বুঝি
- [] Higher-Order Function explain করতে পারি
- [] Callback এবং HOF আলাদা করতে পারি
- [] Function composition বুঝি
- [] `compose()` implement করতে পারি
- [] `pipe()` implement করতে পারি
- [] `compose()` vs `pipe()` explain করতে পারি
- [] Currying implement করতে পারি
- [] Partial application বুঝি
- [] Currying vs partial application explain করতে পারি
- [] Array method chaining করতে পারি
- [] Chaining-এর performance implications বুঝি
- [] Reducer concept বুঝি
- [] Transducer-এর core idea বুঝি
- [] কখন transducer দরকার আর কখন দরকার নেই বুঝি
- [] React/Redux-এ immutability apply করতে পারি
- [] ছোট pure functions দিয়ে complex pipeline তৈরি করতে পারি

Final Takeaway

Functional Programming-এর মূল কথা কোনো particular method মুখস্থ করা না।

মূল mindset হলো:

```
Input
↓
Pure Transformation
↓
New Value
↓
Pure Transformation
↓
New Value
↓
Final Result
```

Instead of:

```
Input
↓
Mutation
↓
Global State Change
↓
Another Mutation
↓
Unexpected Result
```

তাই JavaScript-এ FP mindset-এর সবচেয়ে গুরুত্বপূর্ণ foundation:

```
Pure Functions
+
Immutability
+
Higher-Order Functions
+
Composition
+
Controlled Side Effects
```

আর এই foundation-এর উপর:

```
Currying
Partial Application
```

Array Chaining Transducers

এর মতো advanced patterns build হয়।